



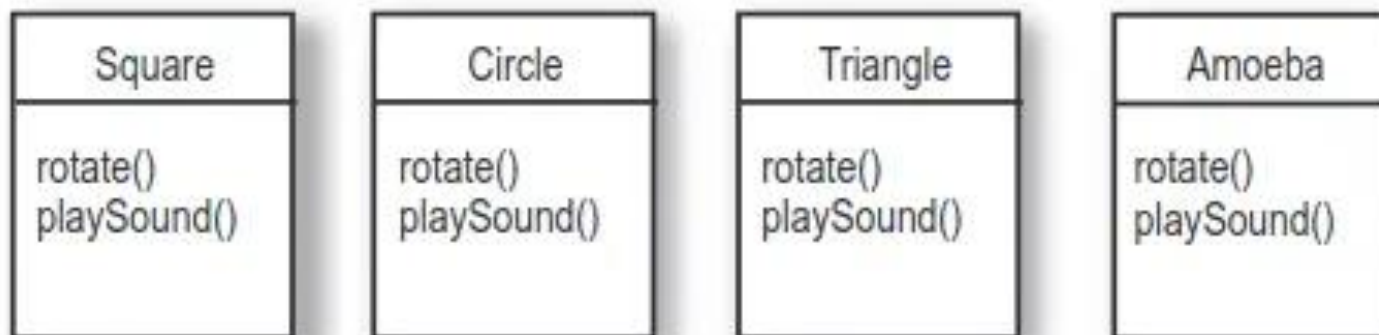
BÀI 06: QUAN HỆ KẾ THỪA GIỮA CÁC LỚP

- ❑ Kế thừa
- ❑ Thiết kế cấu trúc kế thừa
- ❑ Viết đề các phương thức
- ❑ Mối quan hệ IS-A và HAS-A
- ❑ Toán tử móc xích giữa các lớp kế thừa
- ❑ Bao bọc và che giấu thông tin
- ❑ Bài tập



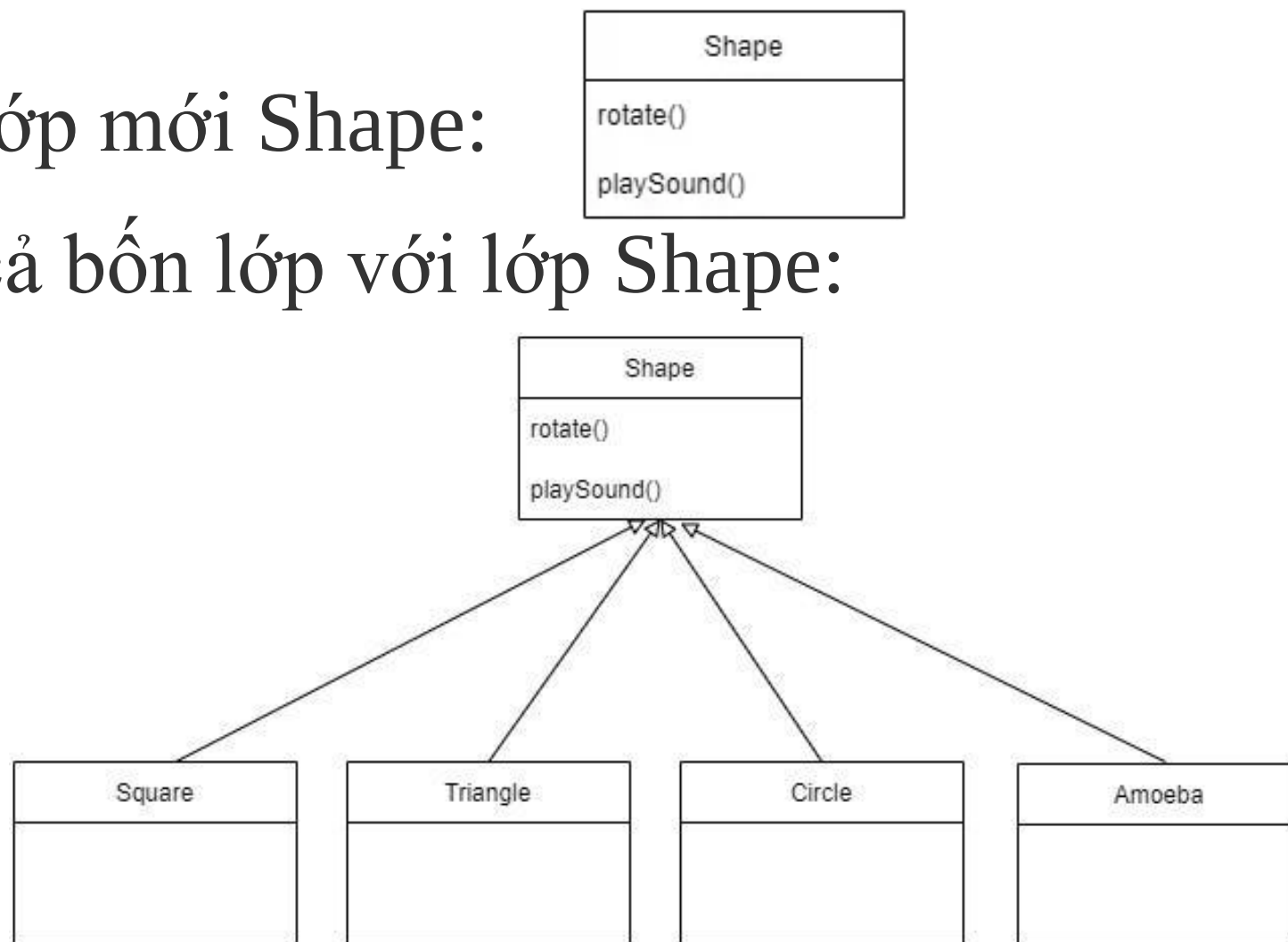
Kế thừa - Ví dụ

- ❑ Giả sử ta phải thiết kế bốn lớp sau



Kế thừa - Ví dụ

- ❑ Thiết kế lớp mới Shape:
- ❑ Liên kết cả bốn lớp với lớp Shape:





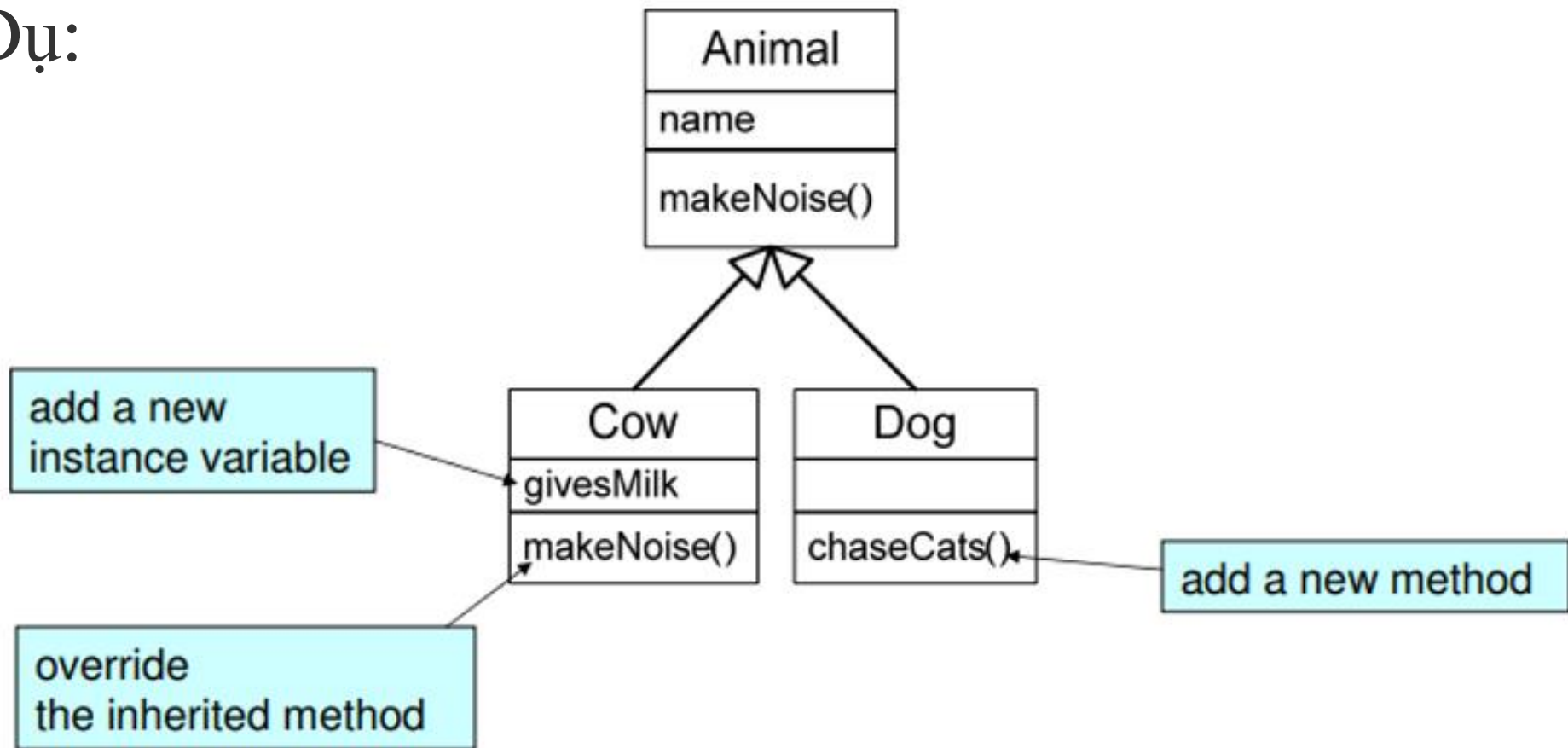
Kế thừa

- ❑ Lớp con kế thừa từ lớp cha, tức là lớp con kế thừa các thành viên của lớp cha:
 - ❑ Thuộc tính
 - ❑ Phương thức
- ❑ Lớp con chuyên biệt hóa lớp cha:
 - ❑ có thể thêm các biến và phương thức mới.
 - ❑ có thể ghi đè lên các phương thức được kế thừa.



Kế thừa

❑ Ví Dụ:





Thiết kế cấu trúc kế thừa

Khai báo:

```
[<Phạm vi>] [<Kiểm soát truy nhập>] class <Tên lớp  
con> [extends <Tên lớp cha>] {  
    // Các thuộc tính dữ liệu bổ sung  
    // Các hàm thành phần bổ sung hay viết đè  
}
```

❑ Ví Dụ:

```
class Animal {  
    String name;  
    void makeNoise() {  
        System.out.print("Hmm");  
    }  
}
```

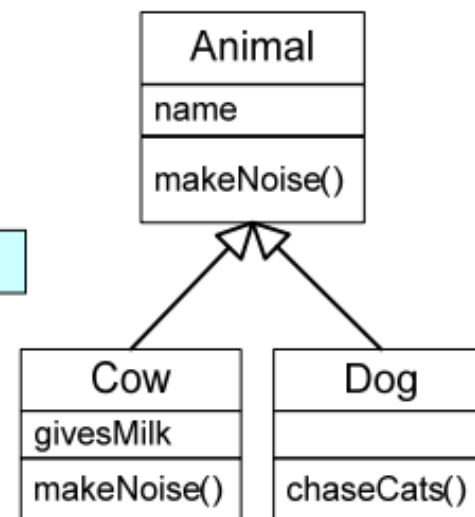
the overridden method

```
class Cow extends Animal {  
    boolean givesMilk;  
    void makeNoise() {  
        System.out.print("Mooooooooooo...");  
    }  
}
```

newly added members

```
class Dog extends Animal {  
    void chaseCats() {  
        System.out.print("I'm coming, cat!");  
    }  
}
```

the inherited method



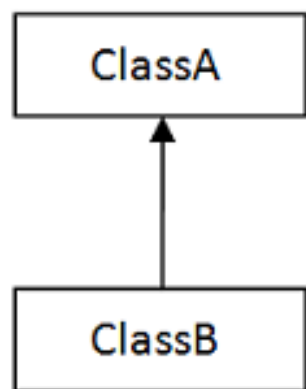
```
Cow cow = new Cow();  
cow.makeNoise();  
cow.givesMilk = true;
```

```
Dog dog = new Dog();  
dog.makeNoise();  
dog.chaseCats();
```

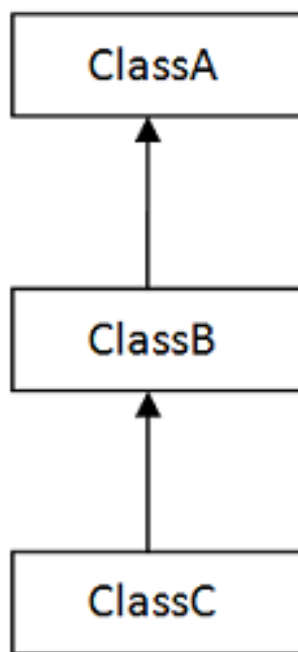


Kế thừa

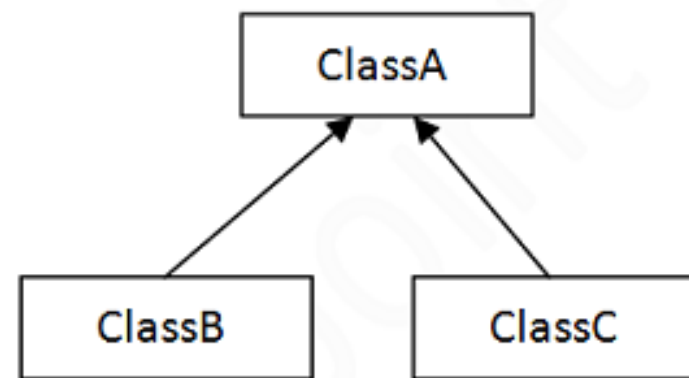
□ Các kiểu kế thừa trong Java:



1. Đơn kế thừa



2. Kế thừa nhiều tầng



3. Kế thừa thứ bậc



Kế thừa

- ❑ Java chỉ hỗ trợ kế thừa đơn (tuyến tính), nghĩa là một lớp chỉ kế thừa được từ một lớp cha.
- ❑ Trong Java lớp cơ sở nhất, làm cơ sở (gốc của cấu trúc phân cấp) cho tất cả các lớp kế thừa là lớp Object.



Thiết kế cấu trúc kế thừa

Bài toán:

- ❑ Giả sử ta cần thiết kế một chương trình giả lập cho phép người dùng thả một đám các con động vật thuộc các loài khác nhau vào một môi trường.
- ❑ Biết rằng mỗi con vật sẽ được đại diện bởi một đối tượng, và các đối tượng sẽ thực hiện các hành vi được lập trình cho loài vật đó.



Thiết kế cấu trúc kế thừa

Bài toán:

- ❑ Danh sách các loài vật sẽ được đưa vào chương trình: sư tử (Lion), hà mã (Hippo), hổ (Tiger), chó (Dog), mèo (Cat), sói (Wolf)



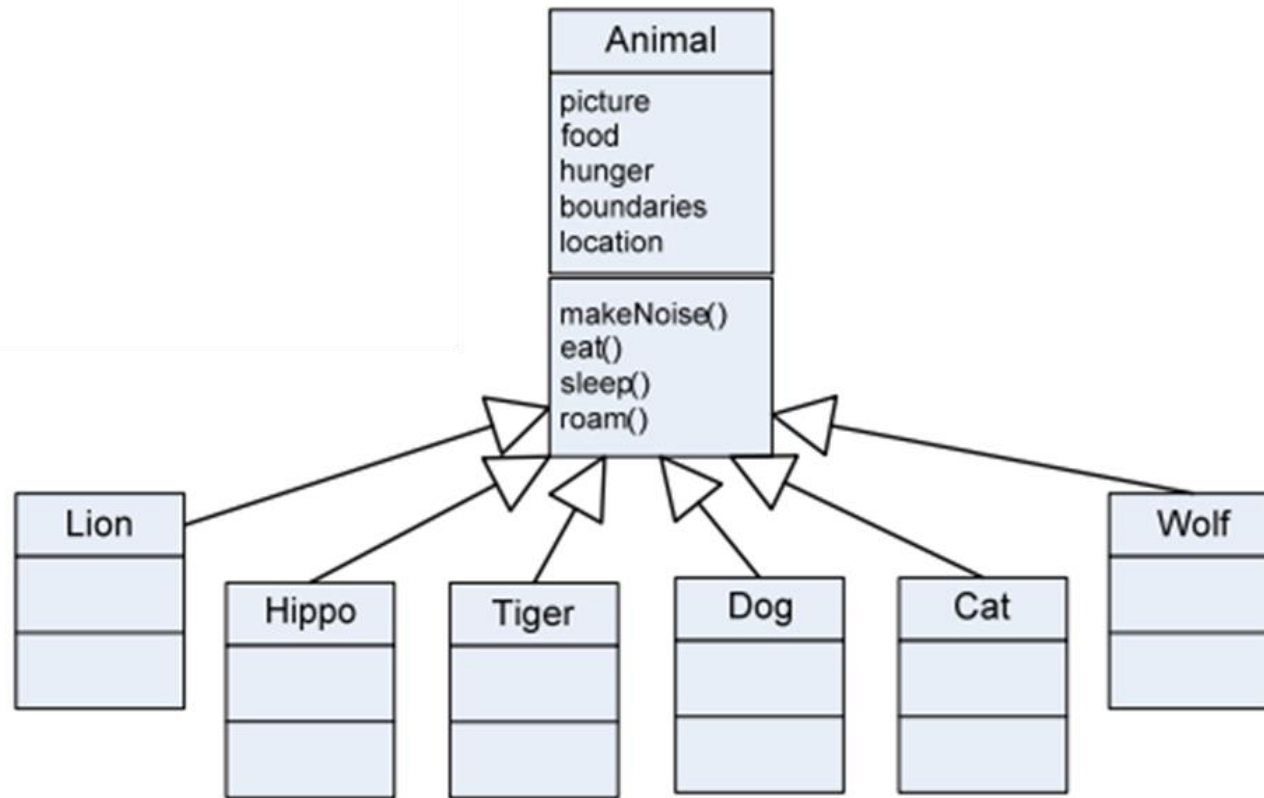
Thiết kế cấu trúc kế thừa

Bước 1: Xác định các đặc điểm chung và trừu tượng mà tất cả các loài động vật đều có:

- ❑ Các thuộc tính: picture, food, hunger, boundaries, location
- ❑ Các phương thức: *makeNoise()*, *eat()*, *sleep()*, *roam()*

Thiết kế cấu trúc kế thừa

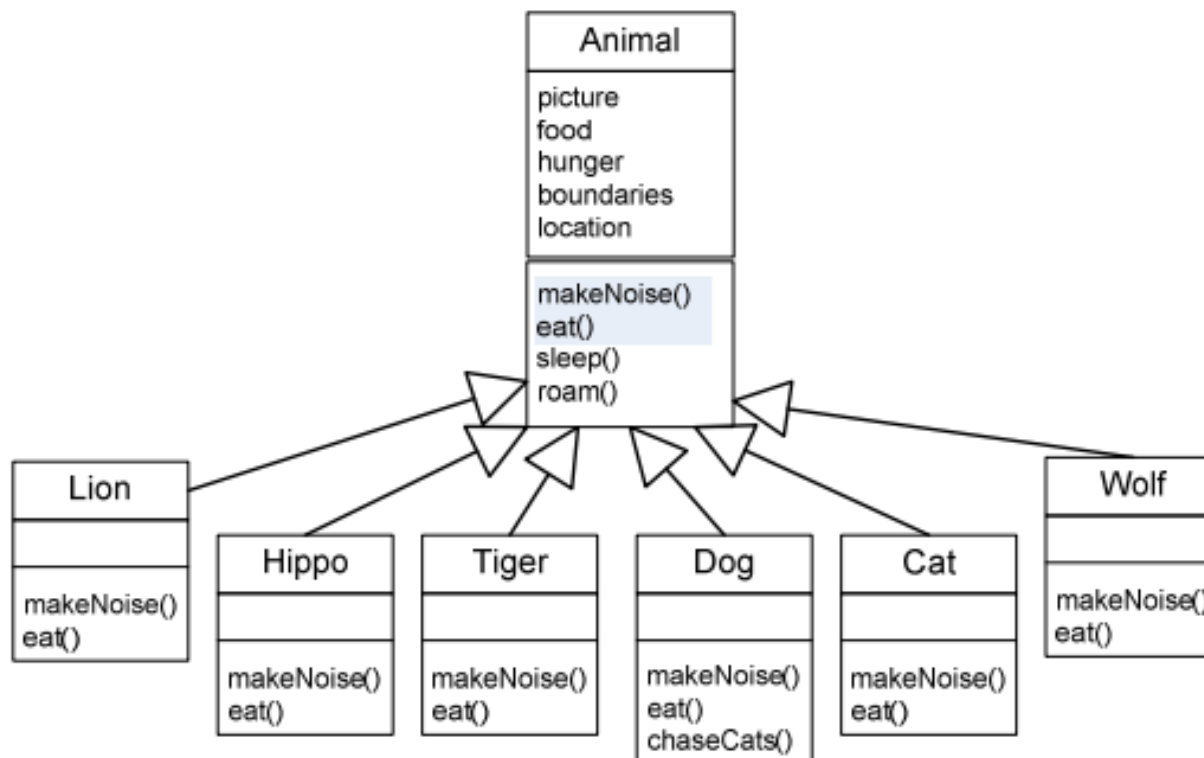
Bước 2: Thiết kế một lớp với tất cả các thuộc tính và hành vi chung (Animal) và xây dựng 1 cây kế thừa cho các loài động vật





Thiết kế cấu trúc kế thừa

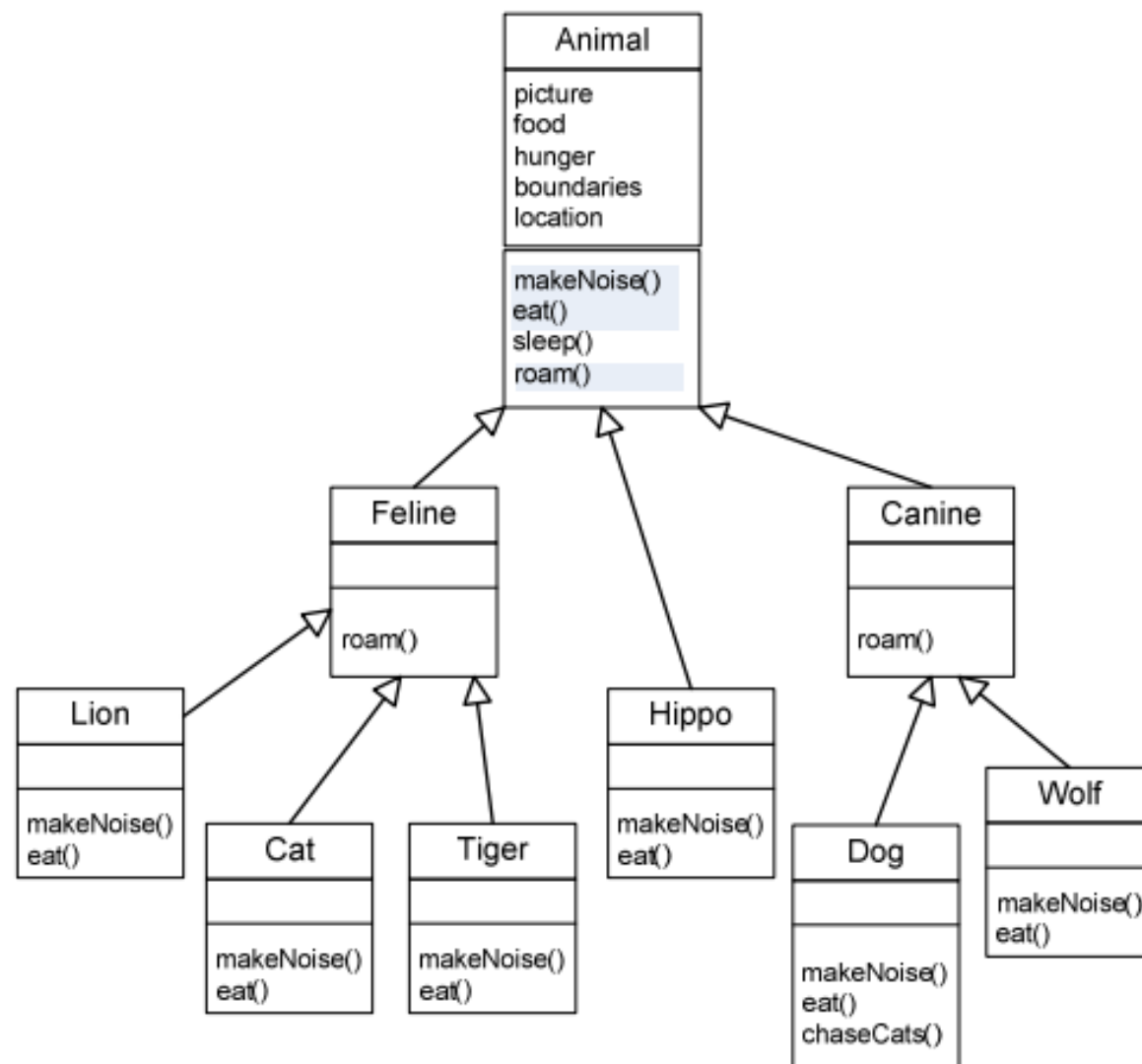
Bước 3: Xác định xem các lớp con có cần các hành vi (phương thức) đặc thù của từng thể loại





Thiết kế cấu trúc kế thừa

Bước 4: Tiếp tục phân nhóm mịn hơn cấu trúc kế thừa trên.





Viết đè các phương thức

- ❑ Một lớp con có thể viết đè, thay đổi nội dung thực hiện của những phương thức được thừa kế từ lớp cha.
- ❑ *Một số lưu ý về cơ chế viết đè hàm thành phần:*
 - ❑ Định nghĩa mới của phương thức viết đè phải có cùng định danh (tên gọi và danh sách tham biến) và cùng kiểu trả lại giá trị.
 - ❑ Phương thức viết đè không được giảm quyền truy nhập so với phiên bản của lớp cha
 - ❑ Định nghĩa của những phương thức sẽ viết đè không được khai báo *final* ở lớp cha.



Viết đề các phương thức

Ví dụ:

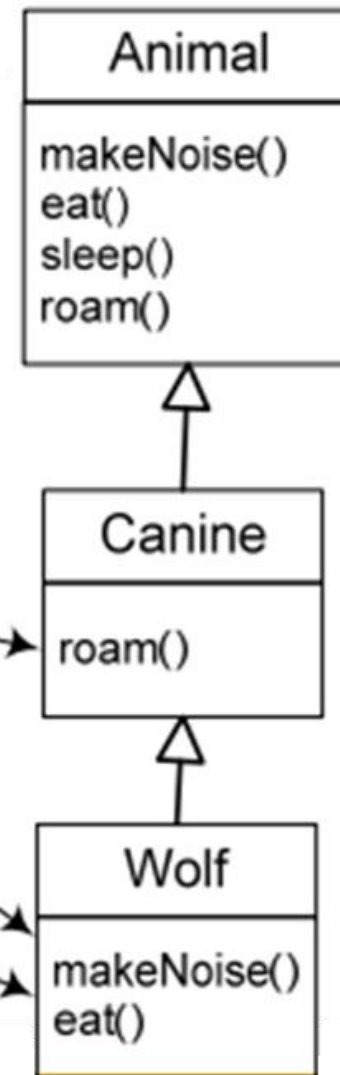
```
Wolf w = new Wolf();
```

```
w.makeNoise();
```

```
w.roam();
```

```
w.eat();
```

```
w.sleep();
```





Mối quan hệ IS-A và HAS-A

- ❑ Vấn đề rất quan trọng trong phát triển phần mềm HĐT là xây dựng được biểu đồ lớp mô tả đầy đủ mối quan hệ giữa các lớp của hệ thống ứng dụng. Hai lớp có thể có các quan hệ sau:
 - ❑ Quan hệ liên kết (*Association*),
 - ❑ Quan hệ kết tập (*Aggregation*),
 - ❑ Quan hệ kế thừa (*Inheritance*),
 - ❑ Quan hệ phụ thuộc (*Dependency*).
- ❑ Hai quan hệ kế thừa và kết tập đều có đặc điểm chung là loại quan hệ giữa “một bộ phận” và phía “tổng thể”. Quan hệ kế thừa là dạng “IS-A” (là một thành viên) còn quan hệ kết tập có dạng “HAS-A” (có một thành viên).



Mối quan hệ IS-A và HAS-A

- ❑ Ví dụ:
 - ❑ Tam giác là một hình (Triangle IS-A Shape)?
 - ❑ Mèo là một động vật họ Mèo (Cat IS-A Feline)?
 - ❑ Xe tải là một phương tiện giao thông (Truck IS-A Vehicle)?

- ❑ Lưu ý: Quan hệ thừa kế IS-A chỉ có một chiều



Mối quan hệ IS-A và HAS-A

- ❑ Ví dụ:
 - ❑ Phòng bếp là một cái nhà (Kitchen IS-A House)?
 - ❑ Nhà có chứa một phòng bếp hay không (House HAS-A Kitchen)?
- ❑ House có một tham chiếu tới một đối tượng Kitchen, chứ House không chuyên biệt hóa Kitchen hay ngược lại

House
Kitchen kitchen; Room bedroom;



Toán tử móc xích giữa các lớp kế thừa

❑ Toán tử this:

- ❑ Được sử dụng để tạo ra đối tượng của lớp hiện thời.
- ❑ Liên kết với đối tượng của lớp hiện thời

❑ Toán tử super:

- ❑ Được sử dụng trong các toán tử tạo lập của lớp con (*subclass*) để gọi tới các toán tử tạo lập của lớp cha (*superclass*) trực tiếp.
- ❑ Liên kết tới đối tượng của lớp cha trực tiếp



Toán tử móc xích giữa các lớp kế thừa

□ *Toán tử this:*

*Tạo ra đối tượng
của lớp hiện thời*

Liên kết với đối tượng
của lớp hiện thời

```
Person(){  
    /*Person(String ht, String dc, int ns){..  
    Person(String ht, String dc){  
        this(ht,dc,2016);  
        System.out.println("Toan tu tao lap su dung voi this");  
    }  
    Person(String hoTen, String diaChi, int namSinh){  
        this.hoTen=hoTen;  
        this.diaChi=diaChi;  
        this.namSinh=namSinh;  
    }  
}
```



Toán tử móc xích giữa các lớp kế thừa

❑ *Toán tử super:*

Gọi tới các toán tử
tạo lập của lớp cha

Liên kết tới đối tượng
của lớp cha trực tiếp

```
class Person{
    private String hoTen;
    private String diaChi;
    private int namSinh;
    Person(){
    }
    /*Person(String ht, String dc, int ns){
    }
    Person(String ht, String dc){
    }
    Person(String hoTen, String diaChi, int namSinh){
    }
    public String nhapXau(){
    }
    public int doiInt(String st){
    }
    public void nhapTT(){
    }
    public void hienThiTT(){
    }
}

public class NhanVien extends Person{
    private int tongLuong;
    NhanVien(){
    }
    NhanVien(String ht, String dc, int ns, int tl){
        super(ht, dc, ns);
        tongLuong=tl;
    }
    public void nhapTT(){
        super.nhapTT();
        System.out.print("Tong luong :"); tongLuong=doiInt(nhapXau());
    }
}
```



Toán tử móc xích giữa các lớp kế thừa

Một số lưu ý khi sử dụng *super* và *this*:

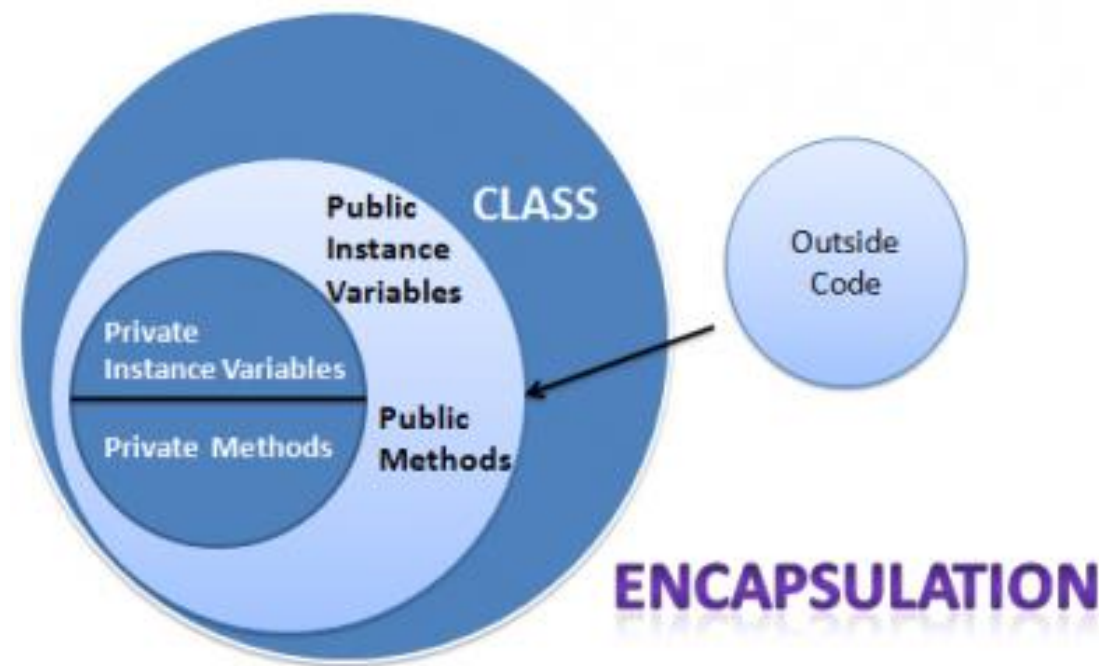
Chúng chỉ sử dụng để xây dựng các toán tử tạo lập của các lớp và khi sử dụng thì chúng luôn phải là lệnh đầu tiên trong định nghĩa của toán tử tạo lập.

this được sử dụng để móc xích với cùng lớp chứa nó còn *super* lại được sử dụng để móc xích với lớp cha của lớp đó.



Bao bọc và che giấu thông tin

Tính bao đóng (Encapsulation) là kỹ thuật ẩn giấu thông tin không được bảo vệ và chỉ hiển thị ra thông tin cho phép truy cập





Bao bọc và che giấu thông tin

- ❑ Để đạt được tính bao đóng (Encapsulation) trong Java chúng ta cần:
 - ❑ Khai báo các thuộc tính của một lớp là private.
 - ❑ Cung cấp phương thức setter và getter là public để có thể sửa đổi và xem các giá trị biến.



Bao bọc và che giấu thông tin

Ví dụ:

```
public class Student {  
    private String name;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```



Bài tập

1) Xây dựng lớp **Ngươi** để quản lý một số thông tin gồm: họ và tên, năm sinh và các phương thức gồm: (3 điểm)

- + Một số toán tử tạo lập
- + Phương thức **nhapTT()** để nhập thông tin cho 1 người
- + Phương thức **hienThiTT()** để hiển thị thông tin của 1 người

2) Xây dựng lớp **SinhVien** kế thừa từ lớp Người, có thêm các thuộc tính bổ sung: lớp, điểm trung bình và các phương thức gồm:

- + Một số toán tử tạo lập
- + Viết đề phương thức **nhapTT()** để nhập thông tin cho 1 sinh viên
- + Viết đề phương thức **nhapTT()** để hiển thị thông tin của 1 sinh viên

3) Xây dựng lớp **QL_SinhVien** có các phương thức:

- + Nhập vào danh sách gồm m ($2 \leq m \leq 60$) sinh viên
- + Hiển thị danh sách các sinh viên có điểm trung bình lớn hơn 8.5



BÀI 06: QUAN HỆ KẾ THỪA GIỮA CÁC LỚP

- ❑ Kế thừa
- ❑ Thiết kế cấu trúc kế thừa
- ❑ Viết đề các phương thức
- ❑ Mối quan hệ IS-A và HAS-A
- ❑ Toán tử móc xích giữa các lớp kế thừa
- ❑ Bao bọc và che giấu thông tin
- ❑ Bài tập



Sinh viên tự học

- ☐ Làm bài tập trắc nghiệm trên hệ thống LMS
- ☐ Bài tập kỹ năng (nếu có)



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM